

La Visibilidad ante Kubernetes Governa el Failover: un Estudio Empírico de CloudNativePG bajo Inyección de Fallos

Angel A. Parejo R.

Universidad de Carabobo, Valencia, Estado Carabobo, Venezuela
angelparejo@gmail.com

Abstract. La interacción entre los operadores de PostgreSQL y el almacenamiento basado en la Container Storage Interface (CSI) en Kubernetes rara vez se evalúa conjuntamente ante fallos. Sobre un marco descriptivo previo —taxonomía, vocabulario $S = (O, K, M, D)$ e invariantes de consistencia, disponibilidad y durabilidad— este trabajo presenta un estudio empírico de CloudNativePG mediante inyección de fallos en un clúster productivo, confrontando lo anticipado con lo observado bajo eliminación del primario (F1), indisponibilidad sostenida (F2) y partición de red (F3). El hallazgo central: la variable que gobierna el failover no es el fallo, sino su visibilidad ante Kubernetes. Eliminar el pod primario dispara la promoción de una réplica (RTO mediano 7.91 s), pero hacerlo fallar de forma sostenida no promueve ninguna —el operador recrea el pod en su sitio (36.75 s)—, refutando la expectativa inicial; la partición preserva la consistencia sin promover. Ninguna transacción confirmada se perdió (RPO nulo).

Keywords: Kubernetes, PostgreSQL, CloudNativePG, CSI, inyección de fallos, sistemas distribuidos, consistencia, failover

1 Introducción

Kubernetes se ha consolidado como la plataforma de facto para orquestar cargas en contenedores, incluidas aplicaciones con estado como las bases de datos [1]. Ello ha impulsado los operadores de PostgreSQL, que automatizan el ciclo de vida, la replicación y la conmutación por error (failover) [4,11], y en paralelo la Container Storage Interface (CSI), que estandariza la integración de backends de almacenamiento heterogéneos [2]. La literatura, sin embargo, tiende a analizar estas capas por separado [5,6,8], pese a que la consistencia, la resiliencia y la recuperación ante fallos dependen de su interacción [7]. En un clúster real, la confiabilidad depende del comportamiento conjunto de la lógica del operador, la planificación de Kubernetes, la semántica de replicación y las garantías del almacenamiento: una dependencia multicapa poco explorada, y especialmente relevante bajo fallo.

El argumento de fondo —que la confiabilidad de las cargas con estado en Kubernetes es un problema inherentemente multicapa— y su formalización mediante una taxonomía, el vocabulario $S = (O, K, M, D)$ y un conjunto de invariantes se establecen en un trabajo previo de carácter conceptual [20]. Este artículo no reintroduce ese marco, sino que lo *confronta con la evidencia* mediante un estudio empírico de CloudNativePG con inyección controlada de fallos sobre un clúster productivo, que mide RTO, RPO y visibilidad de transacciones. Sus contribuciones son: (i) un hallazgo central —la variable que gobierna el failover no es el fallo, sino su **visibilidad ante Kubernetes**—; en particular, la indisponibilidad sostenida del primario (**pod-failure**) *no* promueve una réplica, en contra de lo anticipado, refutación que es el resultado más contundente del estudio; y (ii) la propuesta de extender el marco con un predicado de visibilidad $V(\text{fallo}, K)$ —el estado del pod ante el plano de control: eliminado, **NotReady**-recreado o **Ready**— que el modelo aún no captura. El alcance está delimitado por las restricciones del clúster productivo (Sección 4).

2 Trabajos Relacionados

La literatura relevante se agrupa en tres líneas. La primera, sobre orquestación, arranca con sistemas como Borg y Omega, que sentaron las bases de la gestión a gran escala e influyeron en Kubernetes [1]; sobre ellos, CSI estandariza el almacenamiento persistente [2] y el patrón *operator* introduce la reconciliación como mecanismo de control [4]. La segunda, sobre bases de datos distribuidas, ha tratado la consistencia y la tolerancia a fallos: CockroachDB evidencia el papel de la replicación [3], el teorema CAP fija los límites entre consistencia, disponibilidad y tolerancia a particiones [5,6], la taxonomía canónica de la dependabilidad fija el vocabulario de fallos, errores y fallas [7]. La tercera, sobre recuperación y replicación, destaca el papel del Write-Ahead Logging (WAL) en la durabilidad [9,10]; más recientemente, el despliegue de cargas con estado mediante operadores ha ganado atención [11,12], aunque sin caracterizar su comportamiento de failover, consistencia y durabilidad ante fallos inyectados —el foco de este trabajo.

Una cuarta vertiente, metodológica, es la verificación empírica de la resiliencia mediante inyección controlada de fallos. El *chaos engineering* propone experimentar deliberadamente con fallos para exponer debilidades [13]; la inyección dirigida por linaje automatiza la búsqueda de combinaciones de fallos que violan garantías [14]; y los estudios sobre particiones de red en producción caracterizan su frecuencia y severidad [15]. Más cerca de este trabajo, el propio proyecto CloudNativePG incorpora pruebas de caos en su integración continua —elimina repetidamente el pod primario para verificar el failover— pero sin cuantificar su coste ni distinguir comportamientos [18]; y Chen et al. evalúan la resiliencia del plano de orquestación de Kubernetes inyectando fallos a gran escala [19], un objeto distinto del de un operador con estado y su durabilidad transaccional observable. Existen además comparaciones prácticas entre operadores publicadas por la industria [17], limitadas a listas de características sin marco formal ni

diseño experimental de inyección de fallos. La brecha no es, pues, la ausencia de inyección de fallos en Kubernetes, sino la combinación específica de operador de PostgreSQL, almacenamiento CSI y un verificador de transacciones a nivel de commit que cuantifica la durabilidad y la consistencia observables.

3 Marco de Referencia

El estudio se apoya en un marco descriptivo previo [20], del que se recuerdan aquí solo los elementos necesarios para interpretar los resultados. El sistema se modela como una tupla $S = (O, K, M, D)$, donde O es la lógica del operador de PostgreSQL, K el plano de control de Kubernetes, M la capa de almacenamiento (el «medio» de persistencia) gestionada mediante CSI —se usa M y no C para no colisionar con la dimensión de Consistencia del marco— y D el motor de base de datos. Más que una formulación cerrada, el modelo captura las interacciones entre capas y permite razonar sobre cómo una decisión en una capa se propaga al comportamiento global bajo fallo; el comportamiento emergente se denota $I(S) = f(O \times K \times M \times D)$ (Fig. 1). La taxonomía de operadores (cloud-native como CloudNativePG; basados en Patroni como Zalando; híbridos como Crunchy), la de almacenamiento CSI (local, distribuido, en red), las dimensiones de evaluación y la comparación analítica de responsabilidades de detección y recuperación entre capas por tipo de fallo se desarrollan en [20]. El estudio empírico se limita a CloudNativePG por una restricción del entorno (Sección 4) y no mide los demás operadores.

Tres invariantes, definidos en ese marco, caracterizan un comportamiento aceptable bajo fallo y guían las mediciones de la Sección 5: un *invariante de consistencia* —toda transacción confirmada debe permanecer accesible tras la recuperación [5]—; una *métrica de disponibilidad* —el RTO se reporta como cantidad observable por escenario, no como umbral duro, al no haberse declarado un objetivo de nivel de servicio (SLO) a priori [7]—; y un *invariante de durabilidad* —toda operación registrada en el WAL debe ser recuperable tras un fallo [9,10]. Cada uno se operacionaliza mediante métricas observables: RPO, latencia de failover y pérdida de transacciones.

4 Metodología Experimental

4.1 Entorno, aislamiento y alcance

Se ejecutó un experimento sobre un clúster Kubernetes en producción (v1.34.6), con PostgreSQL 16.13 y almacenamiento en red respaldado por el controlador CSI de Huawei 4.10.1 (csi.huawei.com) sobre Fibre Channel (SAN). La StorageClass usó política de retención (**Retain**) y enlace diferido (**WaitForFirstConsumer**); la red se gestionó con Calico 3.31.4. El estudio se realizó sobre CloudNativePG (CNPG) 1.28.0, exponente del enfoque cloud-native.

El alcance a un único operador responde a una restricción del entorno: el clúster es productivo y su equipo de operación no permite instalar operadores

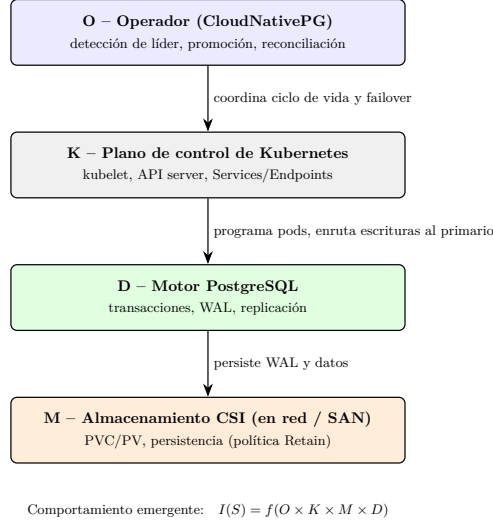


Fig. 1. Modelo de interacción multicapa $S = (O, K, M, D)$: el operador coordina el ciclo de vida y el failover a través del plano de control de Kubernetes; el motor de base de datos persiste el WAL sobre el almacenamiento respaldado por CSI. Adaptado de [20].

nuevos —como Zalando (Patroni/Spilo) o Crunchy— por el riesgo sobre las cargas en producción; el operador CNPG ya presente es, además, compartido con otros clústeres. El contraste con esos operadores se aborda analíticamente en el marco de referencia [20] y se declara trabajo futuro en un entorno no productivo. La inyección de fallos se realizó con Chaos Mesh 2.8.3, compatible con la línea 1.34 de Kubernetes; como el entorno está aislado de redes externas (air-gapped), tanto Chaos Mesh como la imagen del operador se instalaron desde imágenes importadas localmente.

El experimento corrió en un espacio de nombres dedicado, con cuotas y un alcance de inyección restringido por control de acceso, de modo que las cargas productivas quedaron fuera del dominio de fallo. El aislamiento no descansa en la exclusividad del nodo, sino en tres controles: Chaos Mesh restringido al espacio de nombres; un doble filtro de selección —espacio de nombres y nombre de clúster— en cada manifiesto de fallo; y una verificación previa (dry-run) que confirma, por selector, que resuelve exclusivamente a los pods del clúster experimental. El manifiesto solicita anti-afinidad entre instancias, pero el pool de nodos elegibles del laboratorio se reducía a un único worker (`nodo-lab-01`); en consecuencia, las tres instancias (un primario y dos réplicas) co-residieron en ese nodo. El failover observado es, por tanto, intra-nodo, y la tolerancia a un fallo de nodo completo no es testeable por construcción —lo que refuerza la lectura del escenario F2 como cota inferior del RTO ante pérdida de nodo (Secciones 5 y 6).

4.2 Carga de trabajo y cliente de verificación

El operador desplegó un clúster de tres instancias bajo una carga sintética continua generada con `pgbench` (escala `-s 10`, 4 clientes y 2 hilos), en rondas de 60 s que toleran failovers. La replicación se mantuvo en su modo asíncrono por defecto; las cifras de RPO corresponden, pues, a replicación asíncrona. En paralelo, un cliente de verificación (`tx-verifier`) registró cada transacción confirmada y comprobó su visibilidad tras la recuperación: inserta identificadores BIGINT monótonos de un contador propio y anota cada COMMIT con su marca de tiempo. Ante un fallo reintenta el mismo identificador en lugar de avanzar; así los cortes no introducen huecos artificiales y un hueco en la tabla de verdad equivale a una escritura reconocida y perdida, lo que valida la contigüidad de la secuencia como proxy del RPO. La cadencia es un bucle apretado; ante un fallo aplica una espera de 0.2 s antes de reintentar, de modo que la granularidad de medición del RTO —el hueco entre COMMITs— es de ≈ 0.2 s. Como el RTO es un hueco medido con el reloj único del cliente, no depende de la sincronización de relojes entre nodos.

4.3 Escenarios de fallo y tamaño muestral

Se aplicaron tres escenarios reproducibles de forma segura en producción: (i) **F1**, eliminación del pod primario, mediante `PodChaos` en modo `pod-kill`; (ii) **F2**, indisponibilidad sostenida del primario durante 10 min, mediante `PodChaos` en modo `pod-failure`, como aproximación observable de un fallo de nodo desde la perspectiva del operador; y (iii) **F3**, partición de red del primario mediante una `NetworkPolicy` de Calico —sin Chaos Mesh—, aplicada y revertida con `kubectl`. Se planificó un cuarto escenario (F4): un análisis de sensibilidad que inyectaría latencia de E/S creciente (0, 20, 50 y 100 ms) mediante `IOChaos` a nivel de FUSE; resultó no ejecutable sobre CNPG por una interacción entre el inyector FUSE y el endurecimiento de seguridad del operador, y se documenta como lección de instrumentación en la Sección 5.

F1 y F2 se repitieron diez veces cada uno ($n = 10$). F3 comprendió diez repeticiones fijas de 60 s, más dos inyecciones exploratorias de mayor duración (149 s y 300 s) excluidas del cálculo del RTO; la no-promoción se observó en las doce inyecciones (0/12). El tamaño muestral responde a un compromiso entre las ventanas de mantenimiento del clúster productivo y la necesidad de estimar medianas y percentiles; no se realizó un cálculo formal de potencia, y $n = 10$ se declara suficiente para caracterizar tendencia central y dispersión, no para detectar efectos marginales. Dada la distribución no normal esperada de RTO y RPO, se reportan medianas y rangos intercuartílicos. Los fallos de nodo genuinos —desalojo de pods y reasociación (`detach/attach`) de volúmenes— no se reprodujeron: no es posible drenar nodos completos en el clúster productivo, y las tres instancias co-residían en el único nodo elegible. Por ello el escenario F2 debe interpretarse como una cota inferior del RTO real ante pérdida de nodo, no como una réplica completa de dicho evento.

Tabla 1. RTO y RPO por escenario de fallo (CloudNativePG). RTO = tiempo hasta restablecer la escritura, medido como la mediana del hueco entre transacciones confirmadas por el cliente de verificación, con granularidad de instrumento ≈ 0.2 s; RPO = transacciones confirmadas no visibles tras el fallo. «CP» denota el comportamiento observado en que el sistema privilegia la consistencia sobre la disponibilidad (sin split-brain), en sentido operacional y no como afirmación formal sobre CAP. Tamaños muestrales: F1 y F2, $n = 10$; F3, 10 repeticiones fijas más 2 exploratorias excluidas del RTO (no-promoción 0/12). Fuente: mediciones del piloto.

Escenario	¿Promueve? RTO / indisponibilidad	RPO
F1 — pod-kill (eliminación)	Sí 7.91 s (mediana)	0
F2 — pod-failure (sostenida)	No 36.75 s (mediana)	0
F3 — partición de red (Calico)	No = duración de la partición	0
F4 — latencia de E/S (IOChaos FUSE)	— No ejecutable	—

5 Resultados

Esta sección reporta las mediciones obtenidas al ejecutar la metodología sobre CloudNativePG. El análisis es esencialmente descriptivo —medianas, rangos intercuartílicos e intervalos de confianza de mediana— y se complementa con una única prueba de dos grupos, intra-CNPG, que contrasta el RTO de F1 frente al de F2. La Tabla 1 resume el comportamiento observado.

5.1 Tiempo de recuperación (RTO) y pérdida de datos (RPO)

El RPO fue nulo en los tres escenarios, pero su alcance probatorio difiere. Solo F1 pone a prueba la durabilidad ante un failover real: al promover una réplica, un RPO nulo implica que ninguna escritura reconocida sobrevivió únicamente en el primario caído. En F2 y F3 no hubo promoción, de modo que no había divergencia posible entre primario y réplica y el RPO nulo es trivial. Que CloudNativePG preserve $RPO = 0$ en F1 bajo replicación asíncrona es, por tanto, un hallazgo genuino —no una garantía de diseño—, acotado a una carga de escritura modesta: un **pod-kill** podría en principio perder WAL confirmado no propagado, pero bajo esa carga el retardo de replicación era prácticamente nulo en el instante de la eliminación (inferencia de mecanismo, pues no se registró el LSN lag), de modo que la réplica promovida ya tenía persistido todo el WAL reconocido [16]. Además, al residir las tres instancias en el mismo nodo, la replicación no atravesó un enlace inter-nodo; el $RPO = 0$ en F1 no prueba todavía la durabilidad ante una partición o latencia de red inter-nodo reales (Sección 6). Al término del piloto la tabla de verdad resultó contigua —identificadores 1 a 613 253, sin huecos—: ninguna transacción confirmada se perdió. La variabilidad se concentra, pues, en el tiempo de recuperación y no en la durabilidad.

En la eliminación del primario (F1), CloudNativePG ejecutó un failover en las diez repeticiones: el nuevo primario aceptó escrituras con un RTO mediano de 7.91 s (rango intercuartílico $\approx [7.62, 8.01]$ s, rango total 6.61–8.17 s; intervalo

de confianza de la mediana $[6.85, 8.15]$ s). La indisponibilidad sostenida (F2) produjo un comportamiento distinto: en ninguna de las diez repeticiones hubo promoción. En lugar de promover, el operador recreó el pod primario en su sitio —mismo nombre e idéntico PVC— y lo devolvió a **Ready** conservándolo como primario; la ventana de indisponibilidad tuvo una mediana de 36.75 s (rango intercuartílico $\approx [36.2, 37.1]$ s, rango total 35.5–38.3 s; IC de la mediana $[36.05, 37.66]$ s). La partición de red (F3) tampoco disparó promoción: el pod aislado se mantuvo **Ready** durante toda la partición y la pérdida de disponibilidad coincidió con su duración (p. ej., 60 s $\rightarrow$ 60.75 s de mediana), con recuperación inferior a 1 s al retirar la **NetworkPolicy**. Los intervalos de confianza de mediana son de tipo distribución-libre, con cobertura $\approx 97.9\%$.

Por transparencia: la primera inyección de F2 —corrida de validación previa al lote— arrojó una ventana de ≈ 80.5 s y se excluyó de la mediana, de forma simétrica al tratamiento de las inyecciones exploratorias de F3; la exclusión se justifica por un efecto de arranque en frío (caché fría, cola de reconciliación no calentada, primer ciclo de recreación). Dentro del lote de diez se observa una asociación positiva débil entre el índice de repetición y el RTO (Spearman $\rho \approx 0.62$), que no alcanza significancia con $n = 10$ (valor crítico ≈ 0.65 a $\alpha = 0.05$); aun así, la separación entre F1 y F2 (≈ 29 s) es un orden de magnitud mayor que esta variación intra-F2.

De estas mediciones emerge un resultado contraintuitivo: la recuperación tras eliminar el pod primario (F1) es $\approx 4.6\times$ más rápida que tras hacerlo fallar de forma sostenida (F2), pese a tratarse del mismo componente (razón de medianas $4.65\times$: 36.75 s frente a 7.91 s). La separación entre ambas distribuciones es completa: una prueba de Mann–Whitney entre F1 y F2 ($n = 10$ por grupo) arroja $U = 0$, con un valor p exacto bilateral $\approx 1.1 \times 10^{-5}$, correlación rango-biserial 1.00 y sin solapamiento; la diferencia de medianas, estimada mediante Hodges–Lehmann, es de 28.96 s. El mecanismo es que el **pod-kill** elimina el pod y dispara una promoción inmediata, mientras el **pod-failure** induce al operador a esperar y recrear el pod en su sitio antes de restablecer el servicio. La no-promoción en F2 (0/10) y F3 (0/12) admite una cota superior por intervalo binomial exacto (Clopper–Pearson): con 95% de confianza, la probabilidad real de promoción es $\leq 25.9\%$ para 0/10 y $\leq 22.1\%$ para 0/12; no obstante, la conclusión se apoya sobre todo en el mecanismo observado y no únicamente en el conteo.

El hallazgo central que articula los tres comportamientos es que la variable que determina el failover en CloudNativePG no es el fallo en sí, sino su **visibilidad ante Kubernetes**. En F1, la muerte del contenedor deja el pod en **NotReady**; el operador registra el evento «Current primary isn’t healthy, initiating a failover» y promueve en ≈ 7.9 s. En F3, las sondas del kubelet no se bloquean —el failsafe de Calico preserva el tráfico de host—, el pod aislado permanece **Ready**, el operador no emite evento y no promueve: elige consistencia sobre disponibilidad (comportamiento CP) sin split-brain. F2 refina el mecanismo: aunque el pod queda **NotReady**, se recrea con la misma identidad y PVC, y el operador prefiere esperar esa recreación antes que promover. La promoción

de F1 se explica, por tanto, por la **eliminación** del pod —el primario deja de existir— y no por el mero estado `NotReady`.

5.2 Sensibilidad a la latencia de E/S: una lección de instrumentación

El escenario F4 resultó no ejecutable sobre CloudNativePG, y ese bloqueo es en sí un resultado. El inyector FUSE `toda` de Chaos Mesh realiza un `ptrace attach` sobre el proceso de PostgreSQL, pero al montar el sistema de archivos FUSE falla con `Read-only file system (os error 30)` y entra en un ciclo de reintentos. La causa es que CloudNativePG endurece sus pods con `readOnlyRootFilesystem: true` —junto con `runAsNonRoot` y el descarte de capacidades—, mientras `toda` requiere un sistema de archivos raíz escribible. La configuración era correcta (el dry-run resolvió exclusivamente a los pods del clúster experimental); no se trata de un error de configuración, sino de una incompatibilidad entre el inyector FUSE de Chaos Mesh y el `readOnlyRootFilesystem` de CloudNativePG. Se registra estrictamente como una limitación de instrumentación observada con este par concreto de herramientas, no como una propiedad general. La medición cuantitativa de la sensibilidad a la E/S queda como trabajo futuro; dos vías la harían viable: un test confirmatorio que desactive `readOnlyRootFilesystem`, y la limitación de ancho de banda de E/S a nivel de cgroup (`io.max`), que no depende de FUSE ni modifica el sistema bajo prueba.

5.3 Contraste entre el comportamiento anticipado y el observado

El marco de referencia [20] anticipaba, a partir del comportamiento documentado, que el fallo del pod primario dispara la promoción de una réplica: F1 lo confirma —casi trivialmente—, pero el aporte de mayor peso es la refutación en F2, ya expuesta. Los tres escenarios comparten una variable de control que el marco no captura de forma explícita: la visibilidad del fallo ante Kubernetes. De ahí que se proponga extender $S = (O, K, M, D)$ con un predicado $V(\text{fallo}, K)$ que registre el estado del pod primario —eliminado (F1), `NotReady`-recreado (F2) o `Ready` (F3)— como la dimensión que ordena el gradiente de RTO observado. El bloqueo de F4 apunta a una segunda extensión: la observabilidad e inyectabilidad de los fallos dependen del endurecimiento del operador, interacción que el modelo tampoco recoge. No se reclama una validación del marco, sino una caracterización empírica que lo confronta con el comportamiento observado y señala hacia dónde extenderlo.

6 Discusión

La confiabilidad en entornos cloud-native es un fenómeno inherentemente multi-capa: decisiones aparentemente locales —la selección de un operador o de un tipo de almacenamiento— tienen implicaciones globales en RTO, RPO o la visibilidad de las transacciones, un impacto que la literatura en sistemas distribuidos

rara vez atribuye a la orquestación y a abstracciones de almacenamiento como CSI [6,5,7].

El trabajo presenta limitaciones que delimitan el alcance de sus resultados. El estudio empírico cubre un único operador (CloudNativePG) y una única categoría de almacenamiento —CSI en red, SAN sobre Fibre Channel— con hardware homogéneo, por la restricción del entorno productivo; la comparación entre operadores y categorías se aborda analíticamente en el marco de referencia [20]. De forma crítica, el fallo de nodo no se reproduce como tal: las tres instancias co-residían en el único nodo elegible, por lo que el failover observado es intra-nodo. Esa co-localización acota además el RPO nulo de F1 —la replicación no atravesó un enlace inter-nodo, así que no prueba todavía la durabilidad ante latencia de red inter-nodo real—, y F2 no captura la latencia de reasociación (detach/attach) del volumen CSI, dominante en un fallo de nodo genuino; por ello sus cifras son una cota inferior del RTO real. Por último, F4 no fue ejecutable por la interacción entre el endurecimiento del operador y el inyector FUSE.

7 Conclusiones

Este trabajo confrontó empíricamente un marco descriptivo multicapa [20] con el comportamiento observado de CloudNativePG mediante inyección controlada de fallos sobre un clúster productivo. El hallazgo central es que la variable que gobierna el failover no es el fallo en sí, sino su visibilidad ante Kubernetes: la refutación de que la indisponibilidad sostenida del primario promovería una réplica —no lo hizo en ninguna de las diez repeticiones— es su resultado de mayor fuerza probatoria, y motiva extender el marco con un predicado de visibilidad $V(\text{fallo}, K)$. Ninguna transacción confirmada se perdió (RPO nulo), si bien solo F1 pone a prueba la durabilidad ante un failover real. El aporte entrega una lectura operacional —la visibilidad como variable de control del failover— reutilizable para evaluar configuraciones distintas de la aquí medida. Como trabajo futuro se plantea incorporar otros operadores (Zalando/Patroni y Crunchy) en un entorno no productivo, contrastar las categorías de almacenamiento local y distribuido, reproducir un fallo de nodo genuino y medir la sensibilidad del RTO a la latencia de E/S mediante un mecanismo que no dependa de FUSE.

Disponibilidad de datos y código. Los datos limpios de RTO/RPO por escenario y el paquete de ejecución —manifiestos del clúster y de Chaos Mesh, el cliente verificador de transacciones, la carga pgbench y los scripts de análisis, junto con los registros crudos— se facilitan a petición. El experimento se ejecutó sobre un clúster productivo bajo acceso restringido, por lo que el entorno no es de acceso público; el material permite reproducir el diseño en un clúster equivalente.

References

1. Burns, B., Grant, B., Oppenheimer, D., Brewer, E., Wilkes, J.: Borg, Omega, and Kubernetes. *ACM Queue* **14**(1), 70–93 (2016)

2. Kubernetes: Container Storage Interface (CSI) (2024), <https://kubernetes.io>
3. Taft, A., et al.: CockroachDB: The resilient geo-distributed SQL database. In: Proc. ACM SIGMOD, pp. 1493–1509 (2020)
4. Kubernetes: Operator pattern. Kubernetes Documentation, <https://kubernetes.io/docs/concepts/extend-kubernetes/operator/>. Consultado el 8 de julio de 2026
5. Bailis, P., Ghodsi, A.: Eventual consistency today: limitations, extensions, and beyond. *Communications of the ACM* **56**(5), 55–63 (2013)
6. Gilbert, S., Lynch, N.: Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News* **33**(2), 51–59 (2002)
7. Avizienis, A., Laprie, J.-C., Randell, B., Landwehr, C.: Basic concepts and taxonomy of dependable and secure computing. *IEEE Trans. Dependable and Secure Computing* **1**(1), 11–33 (2004)
8. Khan, W., Kumar, T., Zhang, C., Raj, K., Roy, A.M., Luo, B.: SQL and NoSQL database software architecture performance analysis and assessments—a systematic literature review. *Big Data and Cognitive Computing* **7**(2), art. 97 (2023). <https://doi.org/10.3390/bdcc7020097>
9. Han, F., et al.: PALF: Replicated write-ahead logging for distributed databases. *Proc. VLDB Endowment* **17**(12), 3745–3758 (2024). <https://doi.org/10.14778/3685800.3685803>
10. Stonebraker, M., Kemnitz, G.: The POSTGRES next-generation database management system. *Communications of the ACM* **34**(10), 78–92 (1991)
11. Dobies, J., Wood, J.: Kubernetes Operators: Automating the Container Orchestration Platform. O'Reilly Media, Sebastopol (2020)
12. Mega, C.: Orchestrating information governance workloads as stateful services using Kubernetes operator framework. In: *Service-Oriented Computing (SummerSOC 2023)*. CCIS, vol. 1847, pp. 125–143. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-45728-9_8
13. Basiri, A., Behnam, N., de Rooij, R., Hochstein, L., Kosewski, L., Reynolds, J., Rosenthal, C.: Chaos engineering. *IEEE Software* **33**(3), 35–41 (2016)
14. Alvaro, P., Tymon, S.: Abstracting the geniuses away from failure testing. *Communications of the ACM* **61**(1), 54–61 (2018)
15. Alquraan, A., Takruri, H., Alfatafta, M., Al-Kiswany, S.: An analysis of network-partitioning failures in cloud systems. In: *Proc. 13th USENIX OSDI*, pp. 51–68 (2018)
16. CloudNativePG: CloudNativePG Documentation (2026), <https://cloudnative-pg.io/docs/>. Consultado el 7 de julio de 2026
17. Portworx: Choosing a Kubernetes operator for PostgreSQL, <https://portworx.com/blog/choosing-a-kubernetes-operator-for-postgresql/>
18. Drees, F.: Chaos testing the CloudNativePG project. CloudNativePG Blog, 5 de enero de 2026, <https://cloudnative-pg.io/blog/lfx-chaos-testing-yash-agarwal/>. Consultado el 8 de julio de 2026
19. Chen, Z., Goudarzi, M., Toosi, A.N.: Resilience evaluation of Kubernetes in cloud-edge environments via failure injection. *arXiv:2507.16109 [cs.DC]* (2025)
20. Parejo R., A.A.: Hacia un análisis multicapa de operadores de PostgreSQL y almacenamiento CSI en Kubernetes: taxonomía, modelo formal y protocolo experimental reproducible. Preprint, Zenodo (2026). <https://doi.org/10.5281/zenodo.XXXXXXX>